

MethodFinder: Machine-Learning-Guided Search for Rubik’s Cube Solving Methods

Caleb Fox
Virginia Tech
Blacksburg, Virginia, USA

Ethan Avery
Virginia Tech
Blacksburg, Virginia, USA

ABSTRACT

Humans solve Rubik’s Cubes using a predefined ordering of pieces, called a method. Existing methods have been discovered by hand, but new methods can be computer generated by selecting a new ordering of pieces. Finding the viability of a computer generated Rubik’s cube method generally very computationally expensive, as each method has to be solved multiple times to measure performance. To fix this problem, we propose an Machine Learning Oracle designed to evaluate methods just based on the features of that method, and rank the quality of a method based off of this. This system will consist of several main components, including a domain-specific-language(DSL) for representing methods, a custom method, solution feature extraction, and a scoring pipeline. For our oracle, Random Forest was used to estimate method quality. Our results show that this Random Forest method outperforms the Linear Regression approach. On top of this, a method discovery pipeline is implemented using the Machine Learning Oracle to find high potential methods.

KEYWORDS

Rubik’s Cube, machine learning, method search, surrogate modeling, Random Forest, DSL

1 INTRODUCTION

There are a countless number of ways to solve the classic Rubik’s cube. Each process to solve the classic puzzle is called a method. Many methods have been discovered and popularized, such as CFOP, Roux, and ZZ. Finding a new method however, can be quite challenging, especially one that can surpass the already known methods. It can also be extremely computationally expensive. Designing efficient methods for solving a Rubik’s Cube is a complex problem that involves balancing multiple competing factors, including total move count, number of steps, and overall structure of the method. The traditional approaches to evaluating methods rely heavily on intuition and manual experimentation, which can be time-consuming and difficult to scale when exploring a large space of possible methods.

In this project, we present a system for automatically generating and evaluating Rubik’s Cube solving methods using machine learning. In our approach, we focus on defining methods from a variety of features and using a machine learning model that can predict how effective that method will be without requiring full solve simulations. This allows us to rapidly estimate the quality of a new methods, while skipping through the intense computations.

To support this, we developed a complete data pipeline that generates synthetic methods, produces solves, and computes evaluation metrics that are combined into a single score. These scores serve as labels for training a predictive model. We initially use linear regression as a baseline approach due to its simplicity and efficiency, while we later pivoted to a Random Forest approach.

The primary goal of this work is not to perfectly predict method performance, but rather to provide a reliable ranking heuristic that distinguishes strong methods from weak ones. We also are not looking to immediately find a new best method, instead we are looking to minimize the time it takes to evaluate a new one, which in the future will allow a much more efficient discovery of new methods.

2 RELATED WORK

There has already been some work on using machine learning with the Rubik’s Cube, as well as using ML to approximate expensive computations. One example is DeepCubeA, which uses deep reinforcement learning to solve the cube [1]. Instead of relying on human-designed solving methods, it learns a function that estimates how close a cube is to being solved. This allows it to guide a search process and eventually find a solution. This shows that machine learning can work well for problems like the Rubik’s Cube, even without explicitly programming in solving strategies.

Our work is also related to research on surrogate models, especially in optimization problems where evaluating a solution is expensive. The paper "Surrogate Models and Ensemble Strategies for Expensive Evolutionary Optimization: An Industrial Case Study" looks at how machine learning models can be used to approximate a costly evaluation function [2]. Instead of fully evaluating every possible solution, a surrogate model can estimate the result, which saves time and computation.

This idea fits well with our project. Running full solve simulations for each cube method takes a lot of time, especially when generating large amounts of data. By training a model to predict scores, we are essentially building a surrogate model that approximates the evaluation process. This lets us quickly estimate which methods are worth exploring further without needing to run the full pipeline every time.

Overall, these works show that machine learning can both handle complex Rubik’s cube problems and reduce computation in large search spaces. Our project builds on these ideas by applying them to method evaluation and optimization instead of direct cube solving.

3 METHOD

MethodFinder is built around a large pipeline that implements generating domain-specific methods, solving the methods using them, and machine learning evaluation. At the highest level, the system

defines candidate solving methods using a DSL. These method definitions are saved as a .dsl file. From here we can quantify each method using its features. This will be discussed in more detail in Section 3.1.

Once the methods are defined, they are passed onto a solver. This solver executes solves using each method. This will provide detailed solve data, including move count and other statistics. From this we can get a good idea of the structural features of this method. We can then store this data and use it to quantify and score each method tested.

Using these extracted features, MethodFinder computes a score that reflect the overall quality of each method. These scores are our "ground truth" labels for training our machine learning model. We use models such as Random Forest and Linear Regression to learn the relationship between features and overall performance.

After training off of the solve data, the model can be used to predict the quality of a method just based off its features, rather than having to compute solve data. This allows the system to rank candidate methods without requiring the large computational load. The best candidates can then be solved more directly to confirm how good they actually are.

Figure 1 shows the high level pipeline for the MethodFinder.

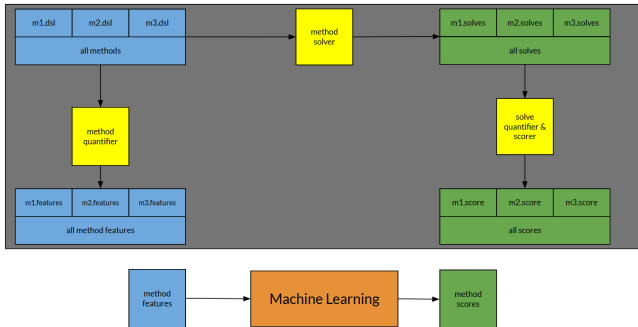


Figure 1: High level MethodFinder pipeline

3.1 Method Solver

The output of the method solver forms the basis for all downstream processing. The collected solve data is later aggregated into statistical features and used to compute method scores, which ultimately serve as labels for training the machine learning model.

The method solver is in charge of executing a given method definition on a scrambled cube and produce a full solve sequence from this. Given the method DSL, the solver takes each step of the method and applies it to the cube. Each method has a sequence of stages, and the solver attempts to find a valid sequence of moves that satisfy each stage. This is done using a search based approach. Figure 2 below shows the general pipeline of the solve.

To make sure that we get enough information from each method, the solver is for many different scrambles for each method. This gives us a better idea about how the score can vary across scrambles. A slightly more in depth look at our method solver shows that we use a program called "kubesolver". We input the scramble and the steps that make up the method, and we get an order of moves. Figure 3 shows this flowchart.

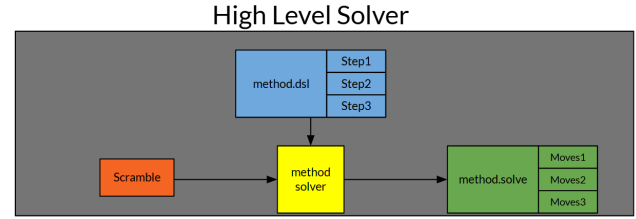


Figure 2: High Level Solver Pipeline

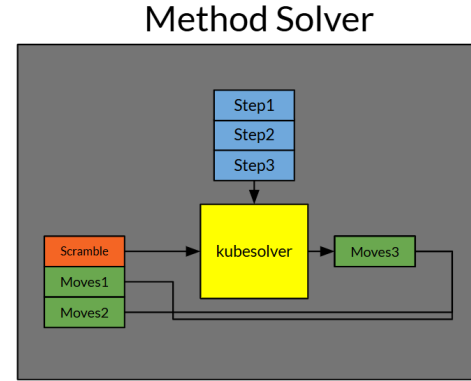


Figure 3: Method Solver

3.2 Method Quantifier

In order to extract the relevant features of each method, a method quantifier was created. While the method solver is able to parse a raw method dsl, this data is not numeric and needs to be transformed for input to a machine learning model. The quantifier is what bridges this gap to get actual statistics from methods.

For any method, the quantifier gets each feature from the method. These are things like number of steps, number of constraints, and more. By starting with the DSL file, the relevant features are extracted by the quantifier and outputted as a vector. An example of a method DSL is shown below for the popular method CFOP

```
[METHOD: CFOP | rotation=x2]
[STEP: cross]
```

```
init_empty_cube
add_edge UF
add_edge UB
add_edge UR
add_edge UL
```

```
[GROUP: F2L | order=best_1]
[STEP: pair_fr]
add_edge BR
add_corner UBR
```

```
[STEP: pair_fl]
add_edge BL
add_corner UBL
```

```

[STEP: pair_b1]
add_edge FL
add_corner UFL

[STEP: pair_br]
add_edge FR
add_corner UFR

[END GROUP]

[STEP: oll | cache_alg=true | free_layer=D]
add_edges_orientation

[STEP: pll | cache_alg=true | free_layer=D]

add_edge FD
add_edge BD
add_edge LD
add_edge RD
add_corner DFR
add_corner DBR
add_corner DBL
add_corner DFL
[END METHOD]

```

Overall, by separating feature extraction into its own stage, the system remains flexible to changes. New metrics can be added or changed with minimal changes to other parts of code, making it overall a very modular system.

3.3 Solve Quantifier

After obtaining solve data for each method, the system will quantify the solve data into a variety of statistics called solves features. This includes things like total times solved, average total moves, number of steps, number of moves per steps, and algorithms required. The entire feature vector is passed to a method scorer function, and a single score value is returned. This uses the solves feature vector and returns a singular score. For each method, all solves are quantified into a feature vector like the one shown below.

```

{
  "method": "CFOP",
  "total_solves": 10,
  "avg_total_moves": 54.3,
  "num_steps": 7,
  "avg_moves_per_step": 7.75,
  "num_algs": 78
}

```

3.4 Machine Learning Models

After quantifying each method and extracting each of their features, the next step is to train an ML model to predict the score of a method based off of its features. Our goal is to take the features extracted from the method and use it to predict the score. The general idea is not to get a perfect score prediction, rather to filter out poor methods and quickly find new methods that work well. From here, more in depth testing can be done to find the true validity of the method.

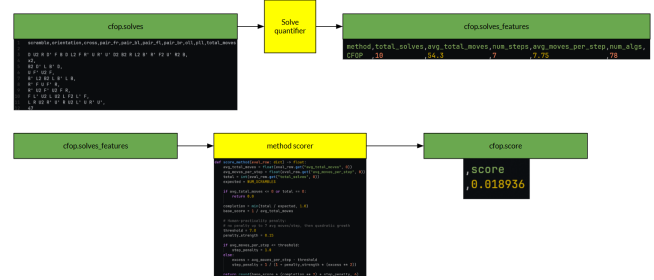


Figure 4: Solve Quantifier Flow Chart

3.4.1 Linear Regression. The first model explored was Linear Regression. Linear Regression was chosen as a baseline because of its simplicity and overall low computational cost. The goal of the model is to take the extracted method features as input and predict that score that represents the overall quality of a solving method.

Linear Regression models the relationship between the input features and the predicted score using a weighted linear combination of the features. This relationship is shown by the function below:

$$h(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n \quad (1)$$

In this equation, each feature x_i represents some property of the method, like average move count or structural complexity. Each corresponding parameter θ_i represents the weight given to that feature by the model. Features with larger weights have a stronger influence on the final predicted score.

The model learns these weights during training using gradient descent. Gradient descent is an algorithm that iteratively adjusts the parameters in order to minimize prediction error across the training dataset. The error is measured using the following cost function:

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m \left(h_{\theta}(x^{(i)}) - y^{(i)} \right)^2 + \frac{\lambda}{2m} \sum_{j=1}^n \theta_j^2 \quad (2)$$

The first term measures the mean squared error between the predicted scores and the actual scores in the dataset. The second term is an $L2$ regularization penalty, which discourages excessively large parameter values. This helps reduce overfitting and improves the model's ability to predict on unknown methods.

Using Linear Regression allows the system to estimate how important each feature is when determining overall method quality. Because the model is linear, the learned weights are also relatively easy to interpret, making it useful for understanding which characteristics most strongly affect performance. However, because many relationships between features are nonlinear, Linear Regression ultimately had limited performance compared to more advanced ensemble methods such as Random Forest.

3.4.2 Random Forest. After running trials with Linear Regression, we found some successful results, though not as much as we had hoped. The Random Forest method was then implemented to compare.

The standard suggested range for Random Forest is 100-500, for this implementation 200 trees were used. These trees grow

Under the current scoring function, the ML-selected candidate achieved a higher score than the baseline methods. This suggests that the search pipeline is able to find promising candidate methods that may perform well.

However, these results should be interpreted carefully. A higher score within this framework does not necessarily mean that the generated method is superior for human solving. The scoring system is primarily based on solver derived statistics such as efficiency and solve structure rather than human usability. Human-centered factors such as memorization difficulty, turn difficulty, recognition speed, and intuitiveness are not captured by the current evaluation pipeline.

The motivation for using machine learning is primarily efficiency. Performing full evaluation on every generated candidate is extremely computationally expensive because each method must be solved repeatedly across many scrambles in order to produce stable

statistics. Fully evaluating 10,000 methods might take longer than 100 days, while evaluating using machine learning methods only takes a few minutes. Once trained, the machine learning model can estimate method quality much more quickly than the full solver. This allows the system to filter out weak candidates early and reserve expensive classical verification for only the most promising methods.

REFERENCES

- [1] F. Agostinelli, S. McAleer, A. Shmakov, and P. Baldi, “Solving the Rubik’s Cube with Deep Reinforcement Learning and Search,” *Nature Machine Intelligence*, 2019. Available: <https://www.nature.com/articles/s42256-019-0070-z> [Accessed: May 10, 2026].
- [2] M. Garza-Fabre, N. Cortés-García, H. Galeana-Zapién, and R. Landa, “Surrogate Models and Ensemble Strategies for Expensive Evolutionary Optimization: An Industrial Case Study,” *IEEE Access*, vol. 12, pp. 86144–86159, 2024. doi: 10.1109/ACCESS.2024.3416811.